

Day 20 - Lecture 20.6: The Complementary Rule

 Python 3.10+

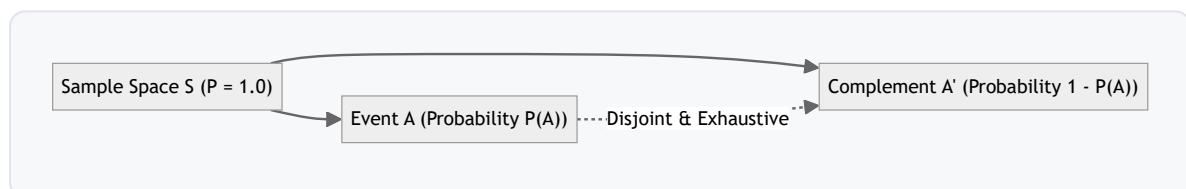
 Jupyter Notebook

PDF notes.pdf

 Language: English | [हिंदी / Hinglish](#) |  [Back to Lecture 20 Module](#)

1. Overview & Conceptual Architecture

The **Complementary Rule** is one of the most powerful analytical shortcuts in probability calculus and computational engineering. For any event A , its complement A' (or A^c) consists of all outcomes in the sample space $\mathcal{S}$ that are not in A .



Strategic Value in Machine Learning:

When calculating the probability of a complex compound event involves dozens of permutations, evaluating its complement is often a trivial single-step calculation:

$$\boxed{P(\text{At least one success}) = 1 - P(\text{Zero successes})}$$

2. Mathematical Formalism & Theorems

By definition, an event A and its complement A' form a binary partition of the sample space:

$$\boxed{A \cap A' = \emptyset \quad \text{and} \quad A \cup A' = \mathcal{S}}$$

Applying Kolmogorov's Axioms 2 and 3:

$$\boxed{P(A \cup A') = P(A) + P(A') = P(\mathcal{S}) = 1.0}$$

Rearranging yields the canonical **Complement Rule**:

$$\boxed{P(A') = 1 - P(A) \text{ iff } P(A) = 1 - P(A')}$$

De Morgan's Laws for Probabilities:

For any two events A and B :

$$\boxed{(A \cup B)' = A' \cap B' \text{ implies } P(A \cup B) = 1 - P(A' \cap B')}$$

$$\boxed{(A \cap B)' = A' \cup B' \text{ implies } P(A \cap B) = 1 - P(A' \cup B')}$$

3. Real-World Case: "At Least One" Failure Problem

Consider an AI server cluster with $n = 10$ independent GPUs. Each GPU has an independent daily failure probability $p = 0.05$. What is the probability that **at least one** GPU fails today?

- Direct calculation: Summing $P(X = 1) + P(X = 2) + \dots + P(X = 10)$ across 10 combinatorial terms.
- Complementary calculation: Let $A = \text{ext}''\textit{AtleastoneGPU fails}''$. Then $A' = \text{ext}''\textit{ZeroGPUs fail}''$. Since all GPUs are independent, $P(\text{ext}GPUsafe) = 1 - 0.05 = 0.95$.

$$\boxed{P(A') = (0.95)^{10} \approx 0.5987}$$

$$\boxed{P(A) = 1 - P(A') = 1 - 0.5987 = 0.4013 \quad (40.13\%)}$$

4. Python Implementation

```
import numpy as np

n_gpus = 10
p_fail = 0.05
p_safe = 1 - p_fail

# Analytical Complement Rule
p_zero_fails = p_safe ** n_gpus
p_at_least_one = 1.0 - p_zero_fails

print(f"Single GPU Failure Rate: {p_fail * 100:.1f}%")
print(f"P(Zero GPU Failures):    {p_zero_fails:.4f}")
print(f"P(At Least 1 Failure):    {p_at_least_one:.4f} ({p_at_least_one * 100:.2f})")

# Monte Carlo Verification (1,000,000 simulations)
simulations = np.random.binomial(n=n_gpus, p=p_fail, size=1_000_000)
empirical_at_least_one = np.mean(simulations >= 1)
print(f"Monte Carlo Empirical:    {empirical_at_least_one:.4f}")
```

5. Key Takeaways & Interview Points

- **The "At Least One" Trigger:** Whenever an interview question or problem asks for "at least one", immediately consider using the complement rule $1 - P(\text{extNone})$.
- **Computational Simplicity:** Complement calculation reduces an exponential combinatorial summation into a single product.